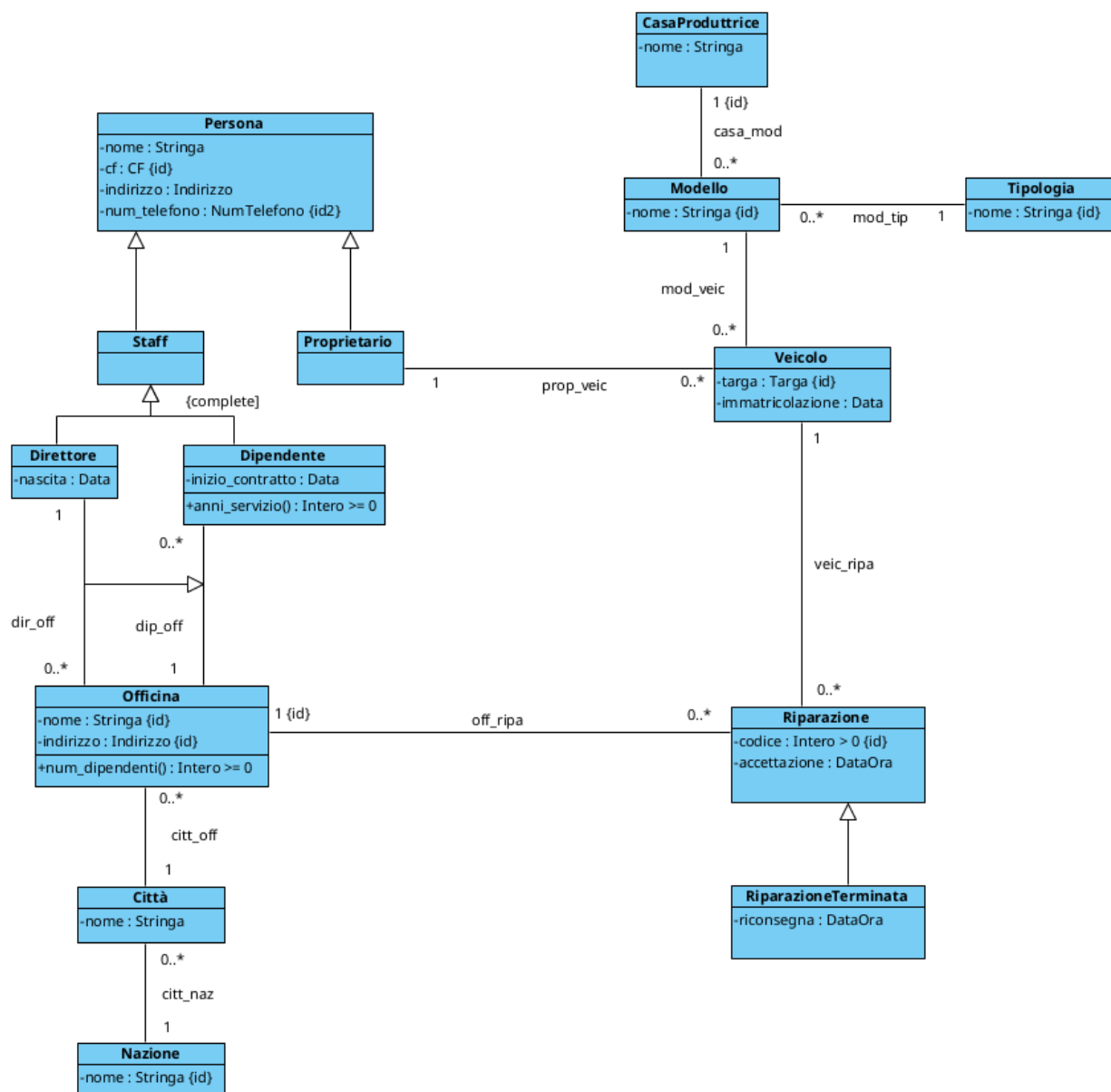


PROGETTAZIONE OFFICINE

Diagramma UML di partenza



FASE 1 - ELIMINAZIONE ATTRIBUTI MULTIVALORE

In fase di analisi non sono stati inseriti attributi multivalore, di conseguenza questa fase non cambia in nessun modo il nostro diagramma.

FASE 2 - SCELTA E PROGETTAZIONE DEI TIPI DI DATO

Andiamo ora a scegliere e creare i nostri tipi di dato SQL.

```
CREATE DOMAIN Stringa AS varchar;
CREATE DOMAIN Data AS date;
CREATE DOMAIN DataOra AS timestamp;

CREATE DOMAIN CF AS varchar(16)
    CHECK (VALUE ~ '^[A-Z]{6}[0-9]{2}[A-Z][0-9]{2}[A-Z][0-9]{3}[A-Z]$')

CREATE TYPE __Indirizzo__ AS (
    Via: varchar(100),
    Civico: int,
    CAP: varchar(5)
)

CREATE DOMAIN Indirizzo AS __Indirizzo__
    CHECK (
        (VALUE).Via IS NOT NULL AND
        (VALUE).Civico IS NOT NULL AND
        (VALUE).CAP ~ '^[0-9]{5}$'
    )

CREATE DOMAIN NumTelefono AS varchar(10)
    CHECK (VALUE ~ '^[0-9]{10}$')

CREATE DOMAIN Targa AS varchar(7)
    CHECK (VALUE ~ '^[A-Z]{2}[0-9]{3}[A-Z]{2}$')

CREATE DOMAIN "Intero > 0" AS int
    CHECK (VALUE > 0)

CREATE DOMAIN "Intero >= 0" AS int
    CHECK (VALUE >= 0)
```

FASE 3 - RISTRUTTURAZIONE DELLE GENERALIZZAZIONI

Scegliamo ora come modificare il diagramma UML per trasformarlo in un diagramma equivalente ma senza generalizzazioni

Generalizzazioni Proprietario e Staff

Per queste due generalizzazioni abbiamo optato per il metodo della sostituzione.

-PRO:

Facilità nel cercare un Cliente (Proprietario) o un membro dello Staff

-CONTRO:

Se si vincola la possibilità di una Persona di avere entrambi i link (disjoint), se un dipendente volesse portare a riparare il suo veicolo allora le sue informazioni sarebbero salvate due volte (una per Proprietario, un'altra per Staff)

Generalizzazioni Direttore, Dipendente e link dir_off

In questo caso abbiamo deciso di accorpate le due classi in una sola, "Dipendente", e di usare il metodo della fusione

-PRO:

Dato che i direttori saranno pochi, non ci preoccupa avere tutti i dati dei dipendenti, direttori o non, all'interno di una sola tabella

-CONTRO:

Molte entuple della tabella Dipendente avranno NULL come valore sulla colonna "nascita", dato che quel dato ci interessa solo se il dipendente è anche direttore

Generalizzazione RiparazioneTerminata

Per questa generalizzazione abbiamo optato per creare due classi, "RiparazioneInCorso" e "RiparazioneTerminata" applicando il metodo della divisione

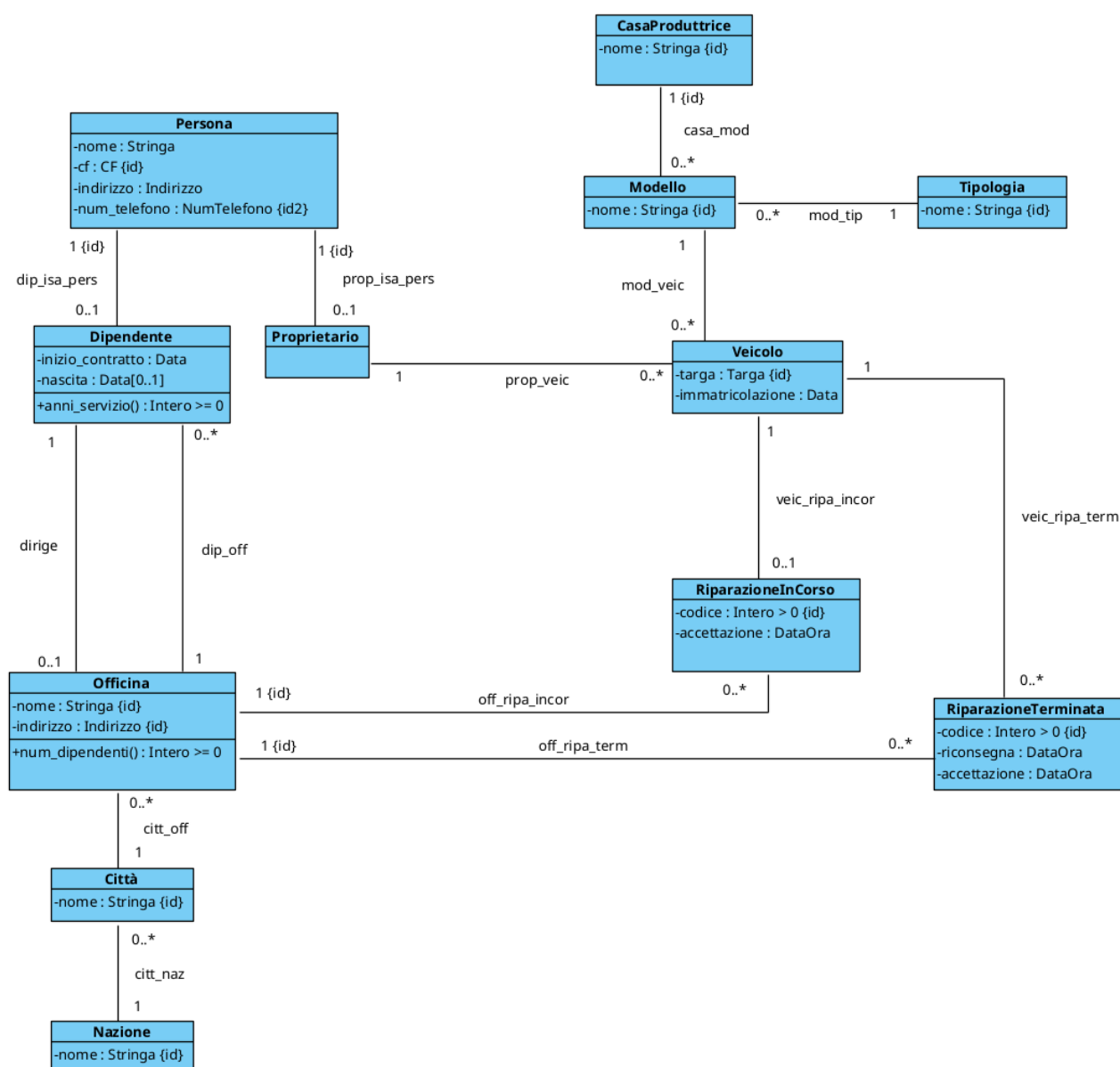
-PRO:

La navigazione tra riparazioni in corso è semplificata

-CONTRO:

Verranno create due tabelle con le informazioni sulle riparazioni, in più, l'aggiornamento dello stato di una riparazione (eliminarla dalla tabella inCorso ed aggiungerla alla tabella Terminate) ha un suo costo

DIAGRAMMA AGGIORNATO



FASE 4 - IDENTIFICATORI PER OGNI CLASSE

Ogni classe ha già un identificatore, quindi questa fase non modifica in alcun modo il diagramma UML

FASE 5 - RISTRUTTURAZIONE VINCOLI ESTERNI ED OPERAZIONI/USE-CASE

Vincoli Esterni

Dato che abbiamo rimosso la generalizzazione Staff, ora abbiamo la classe Dipendente con un attributo opzionale (nascita) proprio perché quell'informazione ci interessa solo se il dipendente è un direttore, quindi va aggiunto il vincolo:

[V.Dipendente.nascita_solo_se_direttore]

```
EXISTS nasc | nascita(this, nasc) <-> EXISTS off | dirige(this, off)
```

Serve anche vincolare che i codici delle riparazioni (sia in corso che terminate) siano univoci (sempre riferiti alla singola officina):

[V.RiparazioneInCorso.no_stesso_codice_riparazioneTerminata]

```
!EXISTS cod, ripaTerm, off |  
  codice(this,cod) and  
  RiparazioneTerminata(ripaTerm) and  
  codice(ripaTerm, cod) and  
  off_ripa_incor(off, this) and  
  off_ripa_term(off, ripaTerm)
```

Operazioni

Le operazioni scritte in fase di analisi non hanno bisogno di nessun cambiamento

Use-Case

Dato che abbiamo rimosso la generalizzazione Riparazione, ora abbiamo le classi RiparazioneInCorso e RiparazioneTerminata da gestire, vanno aggiornati dunque gli use-case:

nuova_riparazioneInCorso(cod:Inter0, v:Veicolo, o:Officina)

```

pre:
  FORALL c, r |
    ( off_ripa_incor(o, r) and codice(r, c) ) -> c != cod
    and
    ( off_ripa_term(o, r) and codice(r, c) ) -> c != cod
  and
  !EXISTS r' |
    veic_ripa_incor(v, r')
  and
  !EXISTS
post:
  il livello estensionale finale differisce
  da quello iniziale come segue:

  nuovi elementi del dominio: alpha
  elementi rimossi dal dominio: nessuno
  nuove ennuple:
    - RiparazioneInCorso(alpha)
    - codice(alpha, cod)
    - accettazione(alpha, adesso)
    - off_ripa_incor(o, alpha)
    - veic_ripa_incor(v, alpha)
  ennuple rimosse: nessuna
  valore di ritorno: result = alpha

```

termina_riparazione(r: Riparazione): RiparazioneTerminata

```

pre:
  !RiparazioneTerminata(r)
post:
  il livello estensionale finale differisce
  da quello iniziale come segue:

  EXISTS off, v |
    off_ripa_incor(off, r)
    veic_ripa_incor(v, r)

  nuovi elementi del dominio: nessuno
  elementi rimossi dal dominio: nessuno
  nuove ennuple:
    - RiparazioneTerminata(r)
    - riconsegna(r, adesso)
    - off_ripa_term(off, r)
    - veic_ripa_term(v, r)

```

```
ennuple rimosse:
  - RiparazioneInCorso(r)
  - off_ripa_incor(off, r)
  - veic_ripa_incor(v, r)
valore di ritorno: result = r
```